

# Geometric SMOTE a geometrically enhanced drop-in replacement for SMOTE

Georgios Douzas, Fernando Bacao\*

NOVA Information Management School, Universidade Nova de Lisboa, Campus de Campolide, Lisboa 1070-312, Portugal

## ARTICLE INFO

### Article history:

Received 16 October 2018

Revised 3 May 2019

Accepted 3 June 2019

Available online 3 June 2019

### Keywords:

Imbalanced learning

Oversampling

Classification

Supervised learning

SMOTE

Data generation

## ABSTRACT

Classification of imbalanced datasets is a challenging task for standard algorithms. Although many methods exist to address this problem in different ways, generating artificial data for the minority class is a more general approach compared to algorithmic modifications. SMOTE algorithm, as well as any other oversampling method based on the SMOTE mechanism, generates synthetic samples along line segments that join minority class instances. In this paper we propose Geometric SMOTE (G-SMOTE) as an enhancement of the SMOTE data generation mechanism. G-SMOTE generates synthetic samples in a geometric region of the input space, around each selected minority instance. While in the basic configuration this region is a hyper-sphere, G-SMOTE allows its deformation to a hyper-spheroid. The performance of G-SMOTE is compared against SMOTE as well as baseline methods. We present empirical results that show a significant improvement in the quality of the generated data when G-SMOTE is used as an oversampling algorithm. An implementation of G-SMOTE is made available in the Python programming language.

© 2019 Published by Elsevier Inc.

## 1. Introduction

Learning from imbalanced data is a non trivial and important problem for the research community and the industry practitioners [9]. An imbalanced learning problem is defined as a classification task for binary or multi-class datasets where a significant asymmetry exists between the number of instances for the various classes. The dominant class is called the majority class while the rest of the classes are called the minority classes [9]. The Imbalance Ratio (IR), defined as the ratio between the majority class and each of the minority classes, depends on the type of application and for binary problems values between 100 and 100.000 have been observed [2,8].

The imbalance learning problem can be found in numerous practical domains, such as chemical and biochemical engineering, financial management, information technology, security, business, agriculture or emergency management, for a more in depth review the reader is referred to (Haixiang et al., 2017). Standard learning methods induce a bias in favor of the majority class during training. This happens because the minority classes contribute less to the minimization of the objective function, defined often as the classification accuracy. Additionally, the distinction between noisy and minority class instances is frequently difficult. As a result the performance of the classifiers, evaluated on metrics suitable for imbalanced data, is low. It is also important to consider that the costs of misclassifying the minority class are frequently much higher than the costs of misclassification of the majority class [13,34]. Diseases screening tests are a typical situation in which false

\* Corresponding author.

E-mail address: [bacao@novaims.unl.pt](mailto:bacao@novaims.unl.pt) (F. Bacao).

negatives involve a much higher cost than the false positives. Therefore, fundamentally the class imbalance challenge is to propose smart and simple ways to improve the accuracy of classifiers for the minority class.

We can classify the approaches to deal with class imbalance into three main groups [18]. The first consists in the modification or creation of algorithms that reinforce the learning towards the minority class. The second is the application of cost-sensitive methods to minimize higher cost errors. The last and more general approach involves the modification at the data level by re-balancing the class distribution. This is usually done through the use of undersampling, oversampling or hybrid methods.

Our focus in this paper is oversampling techniques, which result in the generation of artificial data for the minority class. Synthetic Minority Oversampling Technique (SMOTE) Chawla et al. [8] is the most popular algorithm in this category. SMOTE can be decomposed into two parts: a set of selection rules for the minority class instances and a data generation mechanism once these samples are selected. Specifically, the selection phase constitutes of a process that repeatedly identifies one minority class sample and a random minority class  $k$ -nearest neighbor of it while the data generation mechanism creates synthetic examples along the line segment that joins them. Most existing variations of the SMOTE algorithm modify the selection phase by imposing a set of heuristic rules. Contrary to this, the method proposed in this paper, G-SMOTE, substitutes the data generation mechanism by defining a flexible geometric region around each minority class instance. Then synthetic instances are generated inside the boundaries of the region which are controlled by an appropriate parametrization of the algorithm.

For the evaluation of G-SMOTE as an oversampling method an experimental analysis is performed. The selected imbalanced datasets are publicly available from the UCI [27] and KEEL [1] repositories. In order to test the performance of the algorithm on more extreme cases of imbalance, undersampled modifications of the aforementioned datasets as well as simulated binary class imbalanced data are provided in a total of 69 datasets. Since this work aims to show that G-SMOTE is an enhanced generalization of SMOTE, the experimental procedure includes a comparison between the two algorithms using 4 classifiers and 3 appropriate evaluation metrics. Additionally, applying no oversampling and Random Oversampling are included as baseline methods.

The sections in the paper are organized as follows. In Section 2, an overview of related previous works and existing sampling methods is given. In Section 3, the motivation for G-SMOTE is presented, while Section 4 describes the proposed method in detail. The experimental results as well as conclusions from their analysis are presented in Section 5.

## 2. Related work

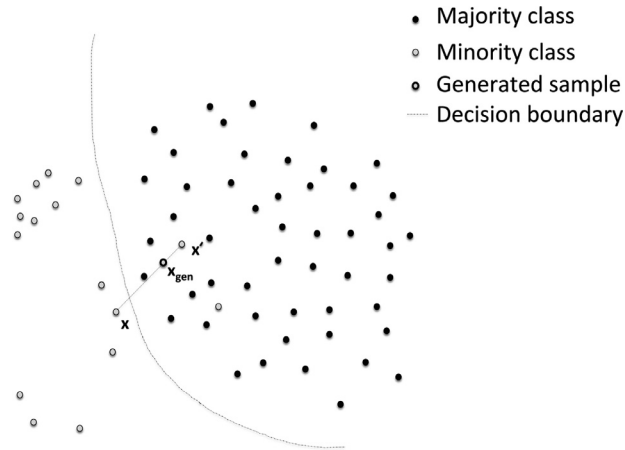
In this section we provide a brief review of the most popular oversampling methods. The reader interested in undersampling and hybrid methods is referred to [7,17] and [20]. The fundamental idea of oversampling methods consists in the generation of synthetic examples for the minority class, which should then be added to the training set. The simplest approach, Random Oversampling, duplicates randomly selected minority class instances. The disadvantage of this approach is that the exact replication of training examples can increase the risk of over-fitting since no new information is created and the classifier will use the same information. An alternative approach that aims to reduce this problem and generate new data is SMOTE, which allows for the generation of synthetic instances along a line segment that joins original minority class instances. SMOTE has been the first and most popular oversampling algorithm. Although SMOTE has been shown to be an effective and simple option for oversampling it also has some weaknesses, such as the fact that the separation between majority and minority class clusters is not often clear and the generation of noisy instances [24]. In order to mitigate these problems many variations to SMOTE have been proposed.

### 2.1. Modifications of the selection phase

Imbalance learning can be divided into two different problems: between-class imbalance and within-class imbalance [25]. The between-class imbalance refers to the classical problem of the skewness in the distribution between majority and minority classes. The within-class imbalance is a subtler, but equally relevant, problem and refers to the possible existence of dense or sparse sub-clusters of minority or majority instances. Both of these problems are relevant in imbalance learning.

SMOTE + Edited Nearest Neighbor Batista et al. [3] is an example of a between-class imbalance algorithm, which modifies the selection phase of the SMOTE algorithm. SMOTE + Edited Nearest Neighbor combination starts by generating artificial instances using SMOTE and then applies the edited nearest neighbor rule [37] to remove misclassified instances, based on the classification by its three nearest neighbors. Borderline-SMOTE [23], MWMOTE (Majority Weighted Minority Oversampling Technique for Imbalanced Data Set Learning) [2], ADASYN and its variation KernelADASYN [33] share the same overall objective, which is to prevent the generation of noisy instances through the identification of the borderline instances for both, majority and minority classes, which, in turn, are used to identify the informative minority class instances.

The typical approach to deal with the within-class imbalance problem is to apply some type of clustering procedure, for the identification of eventual minority or majority sub-clusters, followed by the application of sampling methods, such as SMOTE, to correct the size of different clusters. Cluster-SMOTE [10] applies the  $k$ -means algorithm and then generates artificial data using SMOTE. Similarly DBSMOTE [6] uses DB-SCAN, density-based, algorithm to identify arbitrarily shaped clusters and generates synthetic instances along a shortest path from each minority class instance to a pseudo-centroid of the cluster. A-SUWO [29] creates clusters of the minority class instances with a size, which is determined using cross validation and



**Fig. 1.** An instance near the decision boundary and one of its 4-nearest neighbors are selected randomly. A noisy observation is generated.

generates synthetic instances based on a proposed weighting system. SOMO [15] creates a two dimensional representation of the input space (U-matrix) and based on it, applies the SMOTE procedure to generate intra-cluster and inter-cluster synthetic instances that preserve the underlying manifold structure. Similarly to SOMO, a combination of k-means and SMOTE [16] can be applied to re-balance the class distribution based on the density of the identified clusters. Finally, other types of oversampling approaches are based on ensemble methods [32,36] such as SMOTEBoost [9] and DataBoost-IM [21].

## 2.2. Modifications of the data generation mechanism

The modification or substitution of the SMOTE data generation mechanism is a less common approach. Safe-Level SMOTE [5] applies a weight degree, the safe level, in the data generation process. Based on the safe level, another quantity called the safe level ratio is calculated, that effectively truncates a part of the line segment joining minority class samples. Therefore the Safe-Level SMOTE modification acts as a restriction of the SMOTE data-generation mechanism. A different type of oversampler, that completely substitutes the SMOTE data generation mechanism, applies the Conditional Generative Adversarial Networks (CGAN) [14] to generate data for the minority class. Contrary to the other methods, CGAN oversampler does not rely on local information of the input space but aims to approximate directly the true data distribution.

## 3. Motivation

In the previous section various informative oversampling methods were presented as an effective way to re-balance the data distribution. However, there are scenarios where the SMOTE data generation mechanism may encounter a variety of problems. This section describe some of these cases providing suitable examples and motivates the proposed G-SMOTE algorithm. In what follows  $x$  is the initially selected minority class instance of the SMOTE mechanism. Some of the SMOTE algorithm inefficiencies are the following:

### 1. Generation of noisy instances due to the selection of $k$ -nearest neighbors.

In SMOTE, the value of  $k$  is determined in advance and there are cases where the results of oversampling are sensitive to it as it is shown in the following example. The decision boundary of Fig. 1 identifies areas of the input space where instances from either the positive or negative class dominate. A few of the minority class instances, called noisy observations, that are located near the decision boundary, penetrate into the majority class area. A large  $k$  value can result in the generation of additional noisy examples since  $x'$ , the selected  $k$ -nearest neighbor of  $x$ , might be one of the aforementioned noisy observations.

### 2. Generation of noisy examples due to the selection of an initial observation.

In order to avoid the previous scenario  $k$  is set to a small value. This choice does not eliminate the generation of noisy samples when  $x$  itself is a noisy instance as can be seen in Fig. 2.

### 3. Generation of nearly duplicated instances.

Minority and majority regions can be organized in different clusters, having complex decision boundaries. In this case, being conservative in the selection of  $k$  may be a valid strategy since a small value of it can minimize the probability of generating noisy instances. On the other hand, it may increase the probability of generating synthetic instances in dense minority class areas as is shown in Fig. 3, where  $x$  and  $x'$  belong in to the same cluster. These artificial samples are less useful because they do not add new information to the data set and are conducive to overfitting. Consequently, it is desirable to expand the data generation process in areas where minority examples are absent.

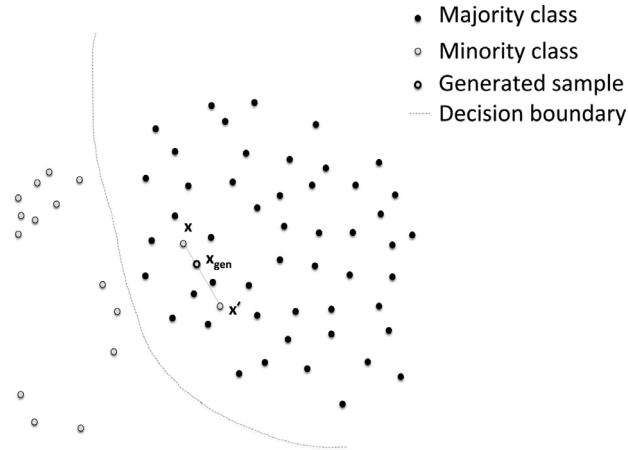


Fig. 2. A 2-nearest neighbor does not avoid the generation of noise when noisy samples are initially selected.

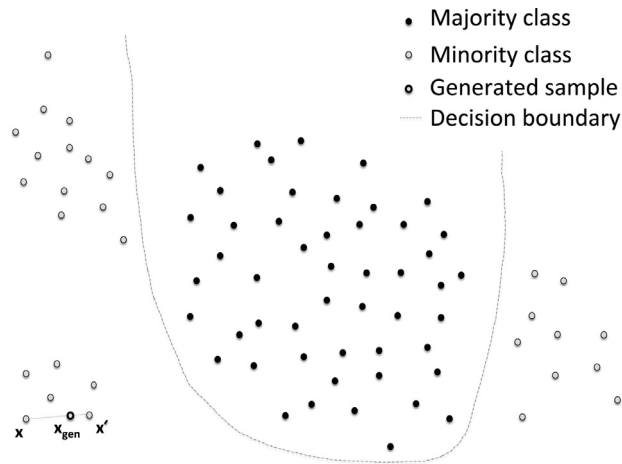


Fig. 3. An instance belonging to a minority class cluster and one of its 5-nearest neighbors are selected. An observation belonging to the same cluster is generated.

#### 4. Generation of noisy instances due to the use of observations from two different minority class clusters.

Increasing  $k$  in order to avoid the previous scenario, may result to a selected  $x'$  such that  $x$  and  $x'$  belong to different clusters. This in turn may lead to the generation of a new instance within the majority region as it is exemplified in Fig. 4.

### 4. The proposed method

In the previous section some insufficiencies of the SMOTE data generation mechanism in various scenarios were described. Some of these insufficiencies apply also to other SMOTE-based oversamplers. We propose a novel data generation procedure, G-SMOTE, which is an extension of the SMOTE algorithm and has three main objectives:

1. To define a safe area around each selected minority class instance such that the generated artificial minority instances inside this area are not noisy.
2. To increase the variety of generated samples by expanding the minority class area.
3. To parametrize the above characteristics based on a small number of transformations with a geometrical interpretation.

G-SMOTE can be considered as a drop-in replacement for SMOTE in the sense that any method relying on the SMOTE data generation mechanism can replace it with the one proposed by G-SMOTE without any further modifications. As it was mentioned on the previous sections, in this paper we aim to a direct comparison of the two data generation mechanisms, with the effect of the above replacement being an investigation of future work.

The complete algorithm in pseudo-code is presented in the following figure:

**Algorithm 1:** G-SMOTE.**Input:**  $S_{maj}, S_{min}, N, k, \alpha_{sel}, \alpha_{trunc}, \alpha_{def}$ **Output:**  $S_{gen}$ **Function** Surface( $\alpha_{sel}, \mathbf{x}_{center}, S_{maj}, S_{min}$ ):

```

  if  $\alpha_{sel} = minority$  then
     $\mathbf{x}_{surface} \in S_{min,k}$ 
  else if  $\alpha_{sel} = majority$  then
     $\mathbf{x}_{surface} \in S_{maj,1}$ 
  else if  $\alpha_{sel} = combined$  then
     $\mathbf{x}_{min} \in S_{min,k}$ 
     $\mathbf{x}_{maj} \in S_{maj,1}$ 
     $\mathbf{x}_{surface} \leftarrow \operatorname{argmin}_{\mathbf{x}_{min}, \mathbf{x}_{maj}} \left( \|\mathbf{x}_{center} - \mathbf{x}_{min}\|, \|\mathbf{x}_{center} - \mathbf{x}_{maj}\| \right)$ 
  return  $\mathbf{x}_{surface}$ 

```

**Function** Vectors( $\mathbf{x}_{center}, \mathbf{x}_{surface}$ ):

```

   $\mathbf{e}_{//} \leftarrow \frac{\mathbf{x}_{surface} - \mathbf{x}_{center}}{\|\mathbf{x}_{surface} - \mathbf{x}_{center}\|}$ 
   $\mathbf{x}_{//} \leftarrow (\mathbf{x}_{gen} \cdot \mathbf{e}_{//}) \mathbf{e}_{//}$ 
   $\mathbf{x}_{\perp} \leftarrow \mathbf{x}_{gen} - \mathbf{x}_{//}$ 
  return  $(\mathbf{x}_{//}, \mathbf{x}_{\perp})$ 

```

**Function** Hyperball():

```

   $v_i \sim N(0, 1)$ 
   $r \sim U(0, 1)$ 
   $\mathbf{x}_{gen} \leftarrow r^{1/p} \frac{(v_1, \dots, v_p)}{\|(v_1, \dots, v_p)\|}$ 
  return  $\mathbf{x}_{gen}$ 

```

**Function** Truncate( $\alpha_{trunc}, \mathbf{x}_{gen}, \mathbf{x}_{center}, \mathbf{x}_{surface}$ ):

```

  if  $|\alpha_{trunc} - \mathbf{x}_{//}| > 1$  then
     $\mathbf{x}_{gen} \leftarrow \mathbf{x}_{gen} - 2\mathbf{x}_{//}$ 
  return  $\mathbf{x}_{gen}$ 

```

**Function** Deform( $\alpha_{dist}, \mathbf{x}_{gen}, \mathbf{x}_{center}, \mathbf{x}_{surface}$ ):

```

  return  $\mathbf{x}_{gen} - \alpha_{def} \mathbf{x}_{\perp}$ 

```

**Function** Translate( $\mathbf{x}_{gen}, \mathbf{x}_{center}, R$ ):

```

  return  $\mathbf{x}_{center} + R \mathbf{x}_{gen}$ 

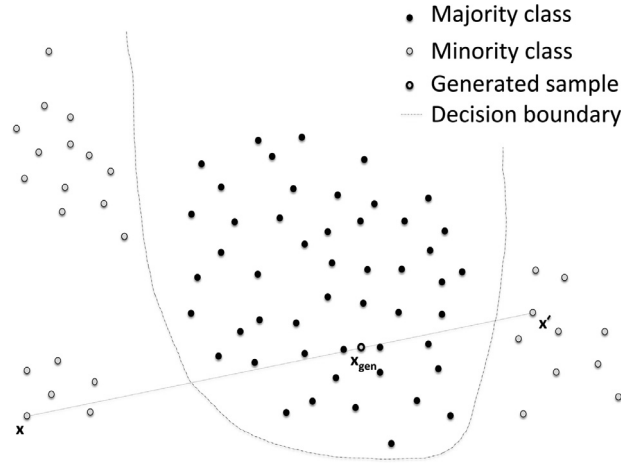
```

**begin**

```

1   $S_{gen} = \emptyset$ 
2  while  $|S_{gen}| < N$  do
3     $\mathbf{x}_{center} \in S_{min}$ 
4     $\mathbf{x}_{surface} \leftarrow \text{Surface}(\alpha_{sel}, \mathbf{x}_{center}, S_{maj}, S_{min})$ 
5     $(\mathbf{x}_{//}, \mathbf{x}_{\perp}) \leftarrow \text{Vectors}(\mathbf{x}_{center}, \mathbf{x}_{surface})$ 
6     $\mathbf{x}_{gen} \leftarrow \text{Hyperball}()$ 
7     $\mathbf{x}_{gen} \leftarrow \text{Truncate}(\alpha_{trunc}, \mathbf{x}_{gen}, \mathbf{x}_{center}, \mathbf{x}_{surface})$ 
8     $\mathbf{x}_{gen} \leftarrow \text{Deform}(\alpha_{dist}, \mathbf{x}_{gen}, \mathbf{x}_{center}, \mathbf{x}_{surface})$ 
9     $\mathbf{x}_{gen} \leftarrow \text{Translate}(\mathbf{x}_{gen}, \mathbf{x}_{center}, \|\mathbf{x}_{center} - \mathbf{x}_{surface}\|)$ 
10    $S_{gen} \leftarrow S_{gen} \cup \{\mathbf{x}_{gen}\}$ 

```



**Fig. 4.** An attempt to generate inter-cluster instances by increasing the number of  $k$ -nearest neighbors of the selected instances. One of the generated instances penetrates in the majority class area.

#### 4.1. G-SMOTE algorithm

The inputs of the G-SMOTE algorithm are the following:

- The sets  $S_{maj}$ ,  $S_{min}$  of majority and minority class samples, respectively.
- The total number  $N$  of synthetic samples to be generated.
- The number  $k$  of nearest neighbors.
- The neighbor selection strategy  $\alpha_{sel}$  with  $\alpha_{sel} \in \{minority, majority, combined\}$ .
- The truncation factor  $\alpha_{trunc}$  with  $-1 \leq \alpha_{trunc} \leq 1$ .
- The deformation factor  $\alpha_{def}$  with  $0 \leq \alpha_{def} \leq 1$ .

The output of G-SMOTE is the set  $S_{gen}$  of generated synthetic examples.

The algorithmic procedure is the following:

- The  $S_{min}$  elements are shuffled and an empty set  $S_{gen}$  is initialized.
- The following loop is repeated until  $N$  minority instances are selected, each multiple times if necessary, in the order that appear in  $S_{min}$ :
  - Let  $\mathbf{x}_{center} \in S_{min}$  the selected minority class instance of  $p$  components.
  - A surface vector is defined from the relation  $\mathbf{x}_{surface} \leftarrow \text{Surface}(\alpha_{sel}, \mathbf{x}_{center}, S_{maj}, S_{min})$ .
  - A set of direction vectors  $(\mathbf{x}_{//}, \mathbf{x}_{\perp}) \leftarrow \text{Vectors}(\mathbf{x}_{center}, \mathbf{x}_{surface})$  is extracted.
  - A synthetic sample  $\mathbf{x}_{gen} \leftarrow \text{Hyperball}$  is generated.
  - The transformation functions *Truncate*, *Deform* and *Translate* are applied to  $\mathbf{x}_{gen}$ .
  - $\mathbf{x}_{gen}$  is added to  $S_{gen}$ .

#### 4.2. Functions

The above algorithmic procedure relies on the following definitions of functions:

- Function *Surface*:

The sets  $S_{min,k}$  and  $S_{maj,1}$  are the sets of  $k$  and  $k = 1$  nearest neighbors of  $\mathbf{x}_{center}$  from  $S_{min}$  and  $S_{maj}$ , respectively.

If  $\alpha_{sel} = minority$  then an element  $\mathbf{x}_{surface} \in S_{min,k}$  is randomly selected.

If  $\alpha_{sel} = majority$  then an element  $\mathbf{x}_{surface} \in S_{maj,1}$  is randomly selected.

If  $\alpha_{sel} = combined$  then  $S_{min,k}$  and  $S_{maj,1}$  are defined as above and the elements  $\mathbf{x}_{min} \in S_{min,k}$  and  $\mathbf{x}_{maj} \in S_{maj,1}$  are randomly selected. Finally the element  $\mathbf{x}_{surface}$  is defined as either  $\mathbf{x}_{min}$  or  $\mathbf{x}_{maj}$  by selecting the one with smallest distance from  $\mathbf{x}_{center}$ .

- Function *Hyperball*:

A vector  $\mathbf{v}_{normal} \leftarrow (v_1, \dots, v_p)$  is generated of  $p$  random numbers from the normal distribution  $N(0, 1)$ . The unit vector

$$\mathbf{e}_{sphere} \leftarrow \frac{\mathbf{v}_{normal}}{\|\mathbf{v}_{normal}\|} \quad (1)$$

and the vector

$$\mathbf{x}_{gen} \leftarrow r^{1/p} \mathbf{e}_{sphere} \quad (2)$$

are calculated where  $r$  is a random number from the uniform distribution  $U(0, 1)$ .

– Function Vectors:

The unit vector

$$\mathbf{e}_{//} \leftarrow \frac{\mathbf{x}_{surface} - \mathbf{x}_{center}}{\|\mathbf{x}_{surface} - \mathbf{x}_{center}\|} \quad (3)$$

and the projection  $x_{//} = \mathbf{x}_{gen} \cdot \mathbf{e}_{//}$  are defined. Using this projection, the vectors

$$\mathbf{x}_{//} \leftarrow x_{//} \mathbf{e}_{//} \quad (4)$$

and  $\mathbf{x}_{\perp} \leftarrow \mathbf{x}_{gen} - \mathbf{x}_{//}$  are also defined.

– Function Truncate:

If the relation

$$|\alpha_{trunc} - x_{//}| > 1 \quad (5)$$

holds then the transformation

$$\mathbf{x}_{gen} \leftarrow \mathbf{x}_{gen} - 2\mathbf{x}_{//} \quad (6)$$

is applied.

– Function Deform:

The transformation

$$\mathbf{x}_{gen} \leftarrow \mathbf{x}_{gen} - \alpha_{def} \mathbf{x}_{\perp} \quad (7)$$

is applied.

– Function Translate:

The transformation

$$\mathbf{x}_{center} + R \mathbf{x}_{gen} \quad (8)$$

is applied.

#### 4.3. Justification of the G-SMOTE algorithm

As explained above, SMOTE compared to Random Oversampling, improves the diversity of generated samples by linearly interpolating generated samples between two minority class instances. However on high-dimensional data SMOTE does not change the class-specific mean values while it decreases the data variability and it introduces correlation between samples [4]. Contrary to this, G-SMOTE extends the linear interpolation mechanism by introducing a geometric region where the data generation process occurs. At the most general choice of hyper-parameters, this geometric region of the input space is a truncated hyper-spheroid. The various steps of the G-SMOTE algorithm can be described in detail as follows:

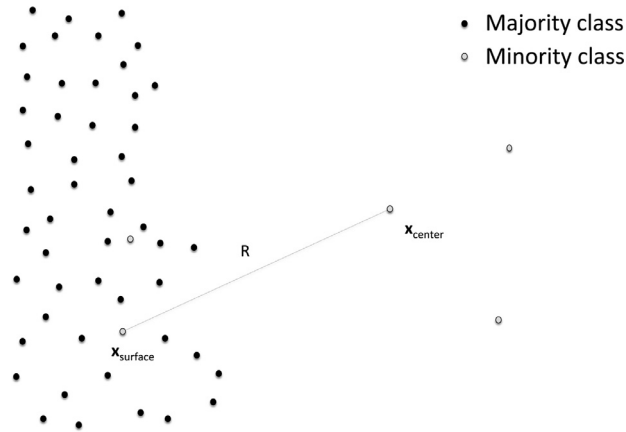
1. An empty set  $S_{gen}$  is initialized.
2. The  $S_{min}$  elements are shuffled and the process described below is repeated  $N$  times until  $N$  artificial points have been generated.
3. A minority class instance  $\mathbf{x}_{center}$  is selected as the center of a geometric region. The order of selection follows the order of dataset points after shuffling on the previous step. Therefore if  $N$  is greater than  $S_{min}$ , then some of the minority class samples will be selected more than once.
4. At first glance, this step generalizes the selection phase of the SMOTE algorithm. More specifically, it results to a randomly selected sample called  $\mathbf{x}_{surface}$  which might belong to either the minority or majority class, depending on the values of  $\alpha_{sel}$ ,  $k$  and  $\mathbf{x}_{center}$ . However, we consider it as a part of the G-SMOTE data generation mechanism since it does not just filters the selected minority class instances based on heuristic rules similarly to SMOTE variations. Based on the neighbor selection strategy  $\alpha_{sel}$ , we distinguish 3 different cases:

(a) Case  $\alpha_{sel} = \text{minority}$ :

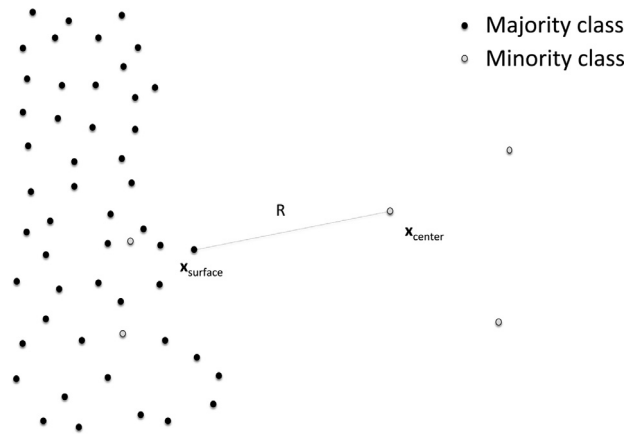
In this case the neighbor selection strategy is based only on the minority class and it is identical to the selection strategy of SMOTE. Initially the  $k$  nearest neighbors of  $\mathbf{x}_{center}$  from the set  $S_{min}$  are identified and one of them,  $\mathbf{x}_{surface}$ , is randomly selected. Fig. 5 presents an example of a minority class instance selection among the  $k = 4$  nearest neighbors of  $\mathbf{x}_{center}$ . The time complexity of this selection strategy depends on the choice of the algorithm and increases with the dimensionality of the input space as well as the value of the  $k$  parameter [35]. Therefore for a wide set of realistic cases restricting the search of nearest neighbors to the minority class has a lower computational cost than including the majority class instances in the search space.

(b) Case  $\alpha_{sel} = \text{majority}$ :

As explained in Section 3, one of the drawbacks of the minority selection strategy is that it may lead to the generation of data penetrating deeply in the majority class area. The majority selection strategy eliminates this scenario. More specifically, the nearest neighbor of  $\mathbf{x}_{center}$  from the set  $S_{maj}$  is identified as  $\mathbf{x}_{surface}$ . The consequence of this selection is that when a random minority class point is generated inside a hypersphere of center  $\mathbf{x}_{center}$  and radius



**Fig. 5.** An example of the minority selection strategy. A minority class instance is defined as the center of the hyper-spheroid and one of its  $k = 4$  minority class nearest neighbors is selected as the surface point. The radius  $R$  of the hyper-spheroid is defined to be equal to the distance of these minority instances.



**Fig. 6.** An example of the majority selection strategy. A minority class instance is defined as the center of the hyper-spheroid and its closest majority class neighbor is selected as the surface point. The radius  $R$  of the hyper-spheroid is defined to be equal to the distance of these instances.

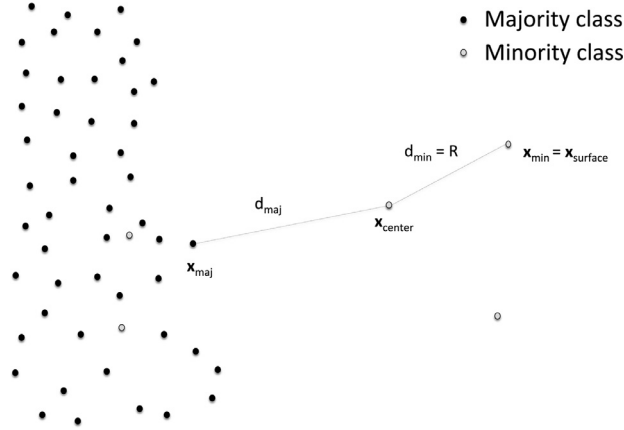
$R = \|\mathbf{x}_{center} - \mathbf{x}_{surface}\|$ , it is ensured that its distance from  $\mathbf{x}_{center}$  is not higher than the distance between  $\mathbf{x}_{center}$  and any majority class instance. On the other hand, since any information about the minority class is discarded, this strategy might aggressively expand the minority class area, resulting effectively to noise generation. Fig. 6 presents an example of the nearest majority class instance selection among the majority class neighbors of  $\mathbf{x}_{center}$ . A disadvantage of the majority selection strategy is that the computational cost compared to the minority selection strategy may be higher, especially for datasets with high IR values.

(c) Case  $\alpha_{sel} = combined$ :

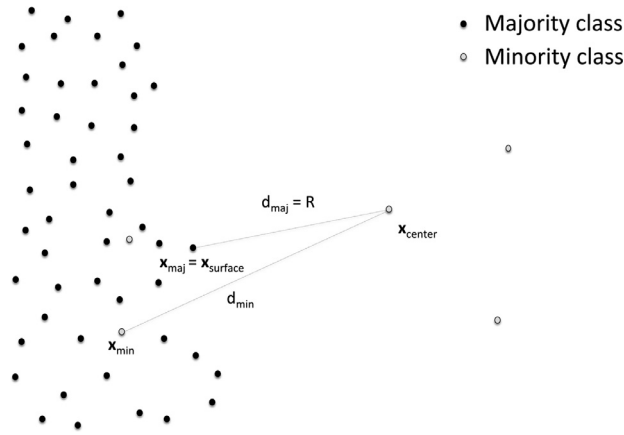
The combined selection strategy initially applies the minority and majority selection strategies, identifying  $\mathbf{x}_{min}$  and  $\mathbf{x}_{maj}$  as the selected minority and majority class instances, respectively. The surface point  $\mathbf{x}_{surface}$  is defined to be either  $\mathbf{x}_{min}$  or  $\mathbf{x}_{maj}$ , so that its distance from the center  $\mathbf{x}_{center}$  is minimized. Figs. 7 and 8 present both of these scenarios i.e. when  $\mathbf{x}_{surface}$  is identified either as a minority or majority class instance. Following the combined selection strategy, the expansion of the minority class area relative to the selected as a center minority class sample is restricted by the nearest majority class neighbor of the center, ensuring that the generation of noisy samples is avoided. Contrary to pure majority selection strategy, the expansion is not only safe but it is further restricted by the presence of minority class instances. The drawback of the combined, similarly to the majority selection strategy, is that it has a higher computational cost compared to the SMOTE/minority selection strategy.

5. Two special directions in the input space are generated:  $\mathbf{x}_{//}$  and  $\mathbf{x}_{\perp}$ . The first one represents the projection of  $\mathbf{x}_{gen}$  to the unit vector  $\mathbf{e}_{//}$  of Eq. (3) that connects  $\mathbf{x}_{center}$  to  $\mathbf{x}_{surface}$ , while the second is perpendicular to the same vector belonging also to the hyperplane defined by  $\mathbf{x}_{gen}$  and  $\mathbf{e}_{//}$ .
6. This step starts the data generation process. A random point  $\mathbf{e}_{sphere}$  is generated on the surface of a unit hyper-sphere centered at the origin of the input space, using Eq. (1). Applying Eq. (2), the point  $\mathbf{e}_{sphere}$  is transformed to a random





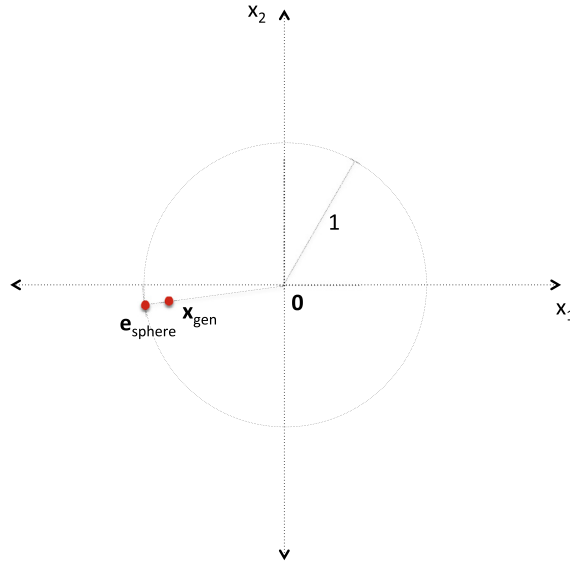
**Fig. 7.** A minority class sample is identified as the surface point since it is closer to the center than the nearest majority class instance.



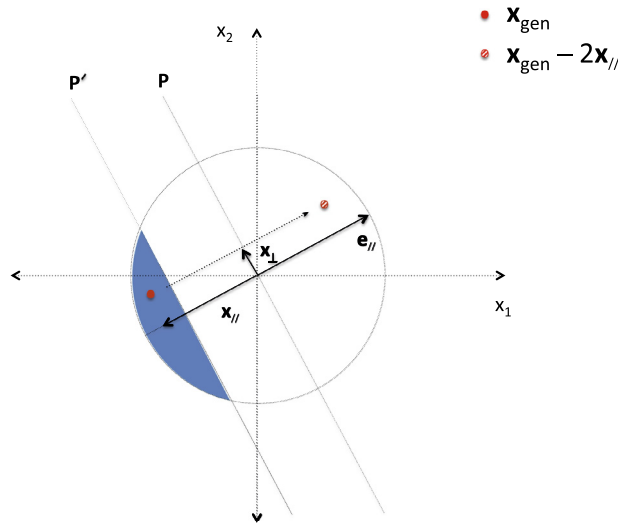
**Fig. 8.** The closest to the center majority class sample is identified as the surface point since it is closer to the center than the selected instance from the  $k$  nearest minority class neighbors of the center.

generated point  $\mathbf{x}_{gen}$  inside the unit hyper-sphere. The final result of this process is a random generated point, uniformly distributed, within the unit hyper-sphere [12]. Fig. 9 shows an example in two dimensions.

7. In this step a transformation is applied to the generated point  $\mathbf{x}_{gen}$ . As it was explained above, the center  $\mathbf{x}_{center}$  and the selected surface point  $\mathbf{x}_{surface}$  define a special direction in the input space which is represented by the unit vector  $\mathbf{e}_{//}$  of Eq. (3). SMOTE mechanism, that always selects a minority class instance as a surface point, exploits this direction by generating synthetic samples at the line segment between  $\mathbf{x}_{center}$  and  $\mathbf{x}_{surface}$ . G-SMOTE algorithm parametrizes a generalized version of the SMOTE mechanism. More specifically, the unit vector  $\mathbf{e}_{//}$  defines a family of parallel hyper-planes which are perpendicular to it. We define a linear mapping between  $\alpha_{trunc}$  and the point determined by the intersection of each hyper-plane and the parallel to  $\mathbf{e}_{//}$  diameter. Therefore each one of these hyper-planes corresponds to a particular value of  $\alpha_{trunc}$  and partitions the hyper-sphere interior in to two areas. Let  $P$  the hyper-plane that passes through the origin and  $P'$  the hyper-plane for a specific non-zero value of  $\alpha_{trunc}$ . When  $\alpha_{trunc} > 0$ , the area that does not include the  $\mathbf{e}_{//}$  point is truncated from the interior of the hyper-sphere, in the sense that if the  $\mathbf{x}_{gen}$  point belongs to it then it is mapped with respect to  $P$  to the symmetric point  $\mathbf{x}_{gen} - 2\mathbf{x}_{//}$  of Eq. (6), where  $\mathbf{x}_{//}$  is defined in Eq. (4). Condition (5) checks if the  $\mathbf{x}_{gen}$  is in the truncated area. Fig. 10 shows an example of the above transformation. When  $\alpha_{trunc} < 0$ , the transformation is similarly defined but in this case the truncation occurs in the area that includes the  $\mathbf{e}_{//}$  point. In both cases, the absolute value of the hyper-parameter  $\alpha_{trunc}$  controls the extent of the truncation. Fig. 11 presents the truncated hyper-sphere areas for various positive and negative values of  $\alpha_{trunc}$ . A final observation is that the above transformation effectively corresponds to a modification of the initial uniform probability distribution in the hyper-sphere. The truncated area acquires a zero value for the p.d.f., while its  $P$ -symmetric mapped area doubles its initial p.d.f. value.
8. This step describes a transformation that corresponds to the deformation of the hyper-sphere in to a hyper-spheroid. More concretely, the point  $\mathbf{x}_{gen}$  is moved to a perpendicular direction to the unit vector  $\mathbf{e}_{//}$ , towards the parallel to  $\mathbf{e}_{//}$  diameter. This mapping is controlled by the  $\alpha_{def}$  hyper-parameter and from Eq. (7) changes linearly with it. Therefore any point located at the surface of the hyper-sphere will remain to the surface of the new boundary while all the axes, except



**Fig. 9.** A unit hyper-sphere centered at the origin of the input space. A point is randomly generated on the surface and moved to the interior of the unit hyper-sphere.



**Fig. 10.** An example of applying the **Truncate** transformation. The shaded area corresponds to the resulting truncated area.

the one defined by the  $\mathbf{e}_{//}$  unit vector, rescale by the factor  $\alpha_{def}$ . This effectively corresponds to the formation of a hyper-spheroid boundary with symmetry axis at the  $\mathbf{e}_{//}$  direction. Similarly to the truncation, the deformation transformation further modifies the initially uniform probability distribution. Fig. 12 presents a deformation of the unit hyper-sphere and the resulting mapping of the  $\mathbf{x}_{gen}$  point. Fig. 13 shows the effect of increasing the  $\alpha_{def}$  values on the hyper-sphere deformation.

9. The final step of the algorithm is the translation of the generated point by the  $\mathbf{x}_{center}$  vector and the rescaling by the value of the radius  $R$ . The combined result of this two transformations is described in Eq. (8). Figs. 14 and 15 show the resulting boundaries of the permissible data generation area as well as a random generated point  $\mathbf{x}_{center}$ , of the two different scenarios presented in Figs. 7 and 8, after the application of truncation, deformation and translation.

## 5. Research methodology

This section describes the evaluation process of G-SMOTE using a variety of classifiers, datasets and metrics. A description of the experimental data, the evaluation measures, the classification algorithms, the experimental procedure as well as the software implementation is provided.

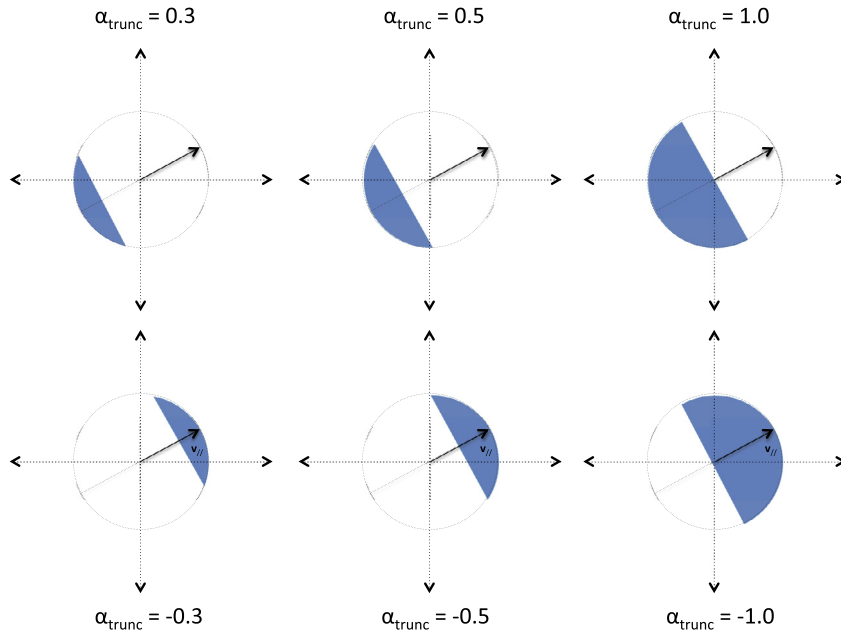


Fig. 11. Truncated areas for various values of  $\alpha_{\text{trunc}}$ .

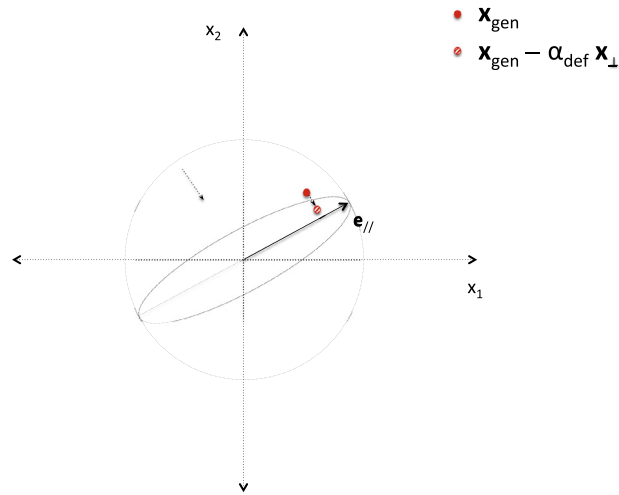


Fig. 12. The transformation *Deform* is applied and the generated point is mapped to a new point towards the diameter of the hyper-sphere.

### 5.1. Experimental data

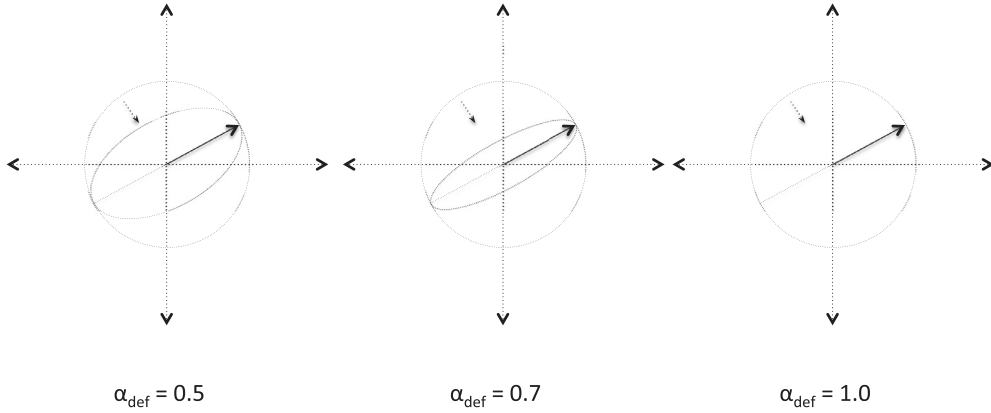
In order to test the performance of G-SMOTE we used the following imbalanced datasets:

- UCI Machine Learning Repository, 13 datasets.
- KEEL repository, 13 datasets.
- Simulated data based on variations of the "MANDELON" dataset [22], 2 datasets.

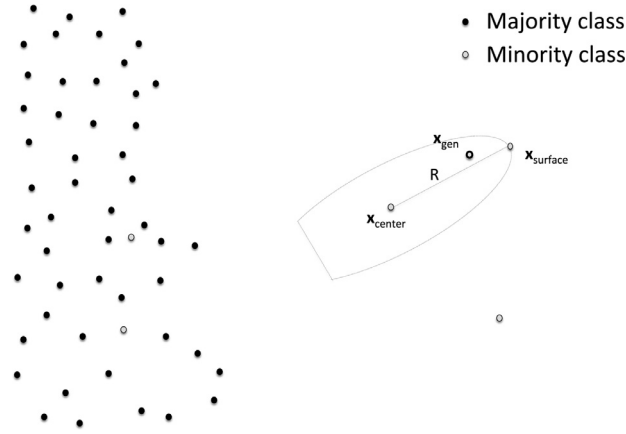
Furthermore, additional datasets with even higher imbalance ratios were generated, by randomly undersampling the minority classes of the aforementioned datasets. For each one of them, its initial IR was multiplied by a factor of 2 and 3 but the resulting dataset was used for the experiments only if the total number of minority samples was not less than 15. Table 1 shows a summary of the final 69 data sets that can be download from [https://www.github.com/IMS-ML-Lab/publications/raw/master/assets/data/imbalanced\\_data.db](https://www.github.com/IMS-ML-Lab/publications/raw/master/assets/data/imbalanced_data.db). The multiplication factor is appended in parentheses to the dataset's names while the table is sorted by ascending IR:

**Table 1**  
Description of the datasets.

| Dataset name      | Features | Instances | Minority instances | Majority instances | Imbalance Ratio |
|-------------------|----------|-----------|--------------------|--------------------|-----------------|
| HEART             | 13       | 270       | 120                | 150                | 1.25            |
| LIVER             | 6        | 345       | 145                | 200                | 1.38            |
| WINE              | 13       | 178       | 71                 | 107                | 1.51            |
| PIMA              | 8        | 769       | 268                | 501                | 1.87            |
| BREASTTISSUE      | 9        | 106       | 36                 | 70                 | 1.94            |
| IRIS              | 4        | 150       | 50                 | 100                | 2.00            |
| GLASS             | 9        | 214       | 70                 | 144                | 2.06            |
| YEAST 1           | 8        | 1,484     | 429                | 1,055              | 2.46            |
| HEART (2)         | 13       | 210       | 60                 | 150                | 2.50            |
| LIVER (2)         | 6        | 272       | 72                 | 200                | 2.78            |
| HABERMAN          | 3        | 306       | 81                 | 225                | 2.78            |
| WINE (2)          | 13       | 142       | 35                 | 107                | 3.06            |
| VEHICLE           | 18       | 846       | 199                | 647                | 3.25            |
| PIMA (2)          | 8        | 635       | 134                | 501                | 3.74            |
| HEART (3)         | 13       | 190       | 40                 | 150                | 3.75            |
| BREAST TISSUE (2) | 9        | 88        | 18                 | 70                 | 3.89            |
| IRIS (2)          | 4        | 125       | 25                 | 100                | 4.00            |
| GLASS (2)         | 9        | 179       | 35                 | 144                | 4.11            |
| LIVER (3)         | 6        | 248       | 48                 | 200                | 4.17            |
| WINE (3)          | 13       | 130       | 23                 | 107                | 4.65            |
| YEAST 1 (2)       | 8        | 1,269     | 214                | 1,055              | 4.93            |
| NEW THYROID 2     | 5        | 215       | 35                 | 180                | 5.14            |
| NEW THYROID 1     | 5        | 215       | 35                 | 180                | 5.14            |
| ECOLI             | 7        | 336       | 52                 | 284                | 5.46            |
| EUCALYPTUS        | 8        | 642       | 98                 | 544                | 5.55            |
| HABERMAN (2)      | 3        | 265       | 40                 | 225                | 5.62            |
| PIMA (3)          | 8        | 590       | 89                 | 501                | 5.63            |
| SEGMENTATION      | 16       | 2,310     | 330                | 1,980              | 6.00            |
| IRIS (3)          | 4        | 116       | 16                 | 100                | 6.25            |
| GLASS (3)         | 9        | 167       | 23                 | 144                | 6.26            |
| VEHICLE (2)       | 18       | 746       | 99                 | 647                | 6.54            |
| YEAST1 (3)        | 8        | 1,198     | 143                | 1,055              | 7.38            |
| YEAST 3           | 8        | 1,484     | 163                | 1,321              | 8.10            |
| HABERMAN(3)       | 3        | 252       | 27                 | 225                | 8.33            |
| PAGE BLOCKS 0     | 10       | 5,472     | 559                | 4,913              | 8.79            |
| VEHICLE (3)       | 18       | 713       | 66                 | 647                | 9.80            |
| VOWEL             | 13       | 988       | 90                 | 898                | 9.98            |
| NEW THYROID 1 (2) | 5        | 197       | 17                 | 180                | 10.59           |
| NEW THYROID 2 (2) | 5        | 197       | 17                 | 180                | 10.59           |
| ECOLI (2)         | 7        | 310       | 26                 | 284                | 10.92           |
| LED               | 7        | 443       | 37                 | 406                | 10.97           |
| EUCALYPTUS (2)    | 8        | 593       | 49                 | 544                | 11.10           |
| SEGMENTATION (2)  | 16       | 2,145     | 165                | 1,980              | 12.00           |
| LIBRAS            | 90       | 360       | 24                 | 336                | 14.00           |
| PAGE BLOCKS 1 3   | 10       | 472       | 28                 | 444                | 15.86           |
| YEAST 3 (2)       | 8        | 1,402     | 81                 | 1,321              | 16.31           |
| ECOLI (3)         | 7        | 301       | 17                 | 284                | 16.71           |
| DERMATOLOGY       | 34       | 358       | 20                 | 338                | 16.90           |
| EUCALYPTUS (3)    | 8        | 576       | 32                 | 544                | 17.00           |
| PAGE BLOCKS 0 (2) | 10       | 5,192     | 279                | 4,913              | 17.61           |
| SEGMENTATION (3)  | 16       | 2,090     | 110                | 1,980              | 18.00           |
| VOWEL (2)         | 13       | 943       | 45                 | 898                | 19.96           |
| LED (2)           | 7        | 424       | 18                 | 406                | 22.56           |
| YEAST 3 (3)       | 8        | 1,375     | 54                 | 1,321              | 24.46           |
| PAGE BLOCKS 0 (3) | 10       | 5,099     | 186                | 4,913              | 26.41           |
| MANDELON 1        | 20       | 4,000     | 142                | 3,858              | 27.17           |
| MANDELON 2        | 200      | 3,000     | 105                | 2,895              | 27.57           |
| YEAST 4           | 8        | 1,484     | 51                 | 1,433              | 28.10           |
| VOWEL (3)         | 13       | 928       | 30                 | 898                | 29.93           |
| YEAST 5           | 8        | 1,484     | 44                 | 1,440              | 32.73           |
| YEAST 6           | 8        | 1,484     | 35                 | 1,449              | 41.40           |
| MANDELON 1 (2)    | 20       | 3,929     | 71                 | 3,858              | 54.34           |
| MANDELON 2 (2)    | 200      | 2,947     | 52                 | 2,895              | 55.67           |
| YEAST 4 (2)       | 8        | 1,458     | 25                 | 1,433              | 57.32           |
| YEAST 5 (2)       | 8        | 1,462     | 22                 | 1,440              | 65.45           |
| MANDELON 1 (3)    | 20       | 3,905     | 47                 | 3,858              | 82.09           |
| MANDELON 2 (3)    | 200      | 2,930     | 35                 | 2,895              | 82.71           |
| YEAST 4 (3)       | 8        | 1,450     | 17                 | 1,433              | 84.29           |
| YEAST 6 (2)       | 8        | 1,466     | 17                 | 1,449              | 85.24           |



**Fig. 13.** The effect of increasing  $\alpha_{def}$  on the hypersphere deformation. The last case corresponds to a line segment hypersphere deformation. The last case corresponds to a line segment.



**Fig. 14.** Boundaries of permissible data generation area for the scenario of Fig 7.

## 5.2. Evaluation measures

Various assessment metrics can be used to evaluate a classifier's performance. However, not all of them are suitable for imbalanced datasets [24]. The most common metric for binary classification problems is accuracy defined as the ratio of correct predictions for both of the classes over the total number of predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where  $TP/TN$  and  $FP/FN$  are the true positives/negatives and false positives/negatives, respectively. When the class distribution is imbalanced, assuming that the minority class is identified as the  $P$  label, the contribution of  $TP$  and  $FP$  to the above formula might be negligible, i.e. accuracy is mainly determined by the majority class contribution. Based on this, the selected evaluation metrics of the experimental procedure are following:

- Area Under the ROC Curve (AUC):

ROC curve results from varying the decision threshold and plotting the true positive rate against the false positive rate.

- F-score:

It is defined as the harmonic mean of *Precision* and *Recall*, where  $\text{Precision} = \frac{TP}{TP+FP}$  and  $\text{Recall} = \frac{TP}{TP+FN}$ .

- G-mean:

It is defined as the geometric mean of *Sensitivity* and *Specificity*, where  $\text{Sensitivity} = \frac{TP}{TP+FN}$  and  $\text{Specificity} = \frac{TN}{TN+FP}$ .

## 5.3. Machine learning algorithms

The main objective of the paper is to compare G-SMOTE and SMOTE algorithms. Therefore their performance was evaluated on the aforementioned 69 datasets. Additionally, the use of unmodified data (NO OVERSAMPLING) and random oversampling (RANDOM OVERSAMPLING) were included as a baseline methods.

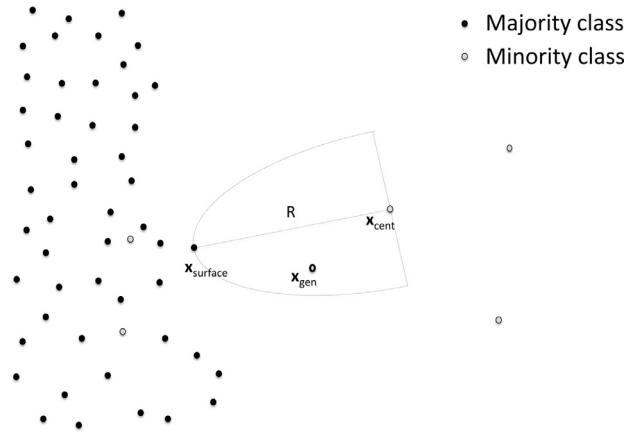


Fig. 15. Boundaries of permissible data generation area for the scenario of Fig 8.

For the evaluation of the oversampling methods the following 4 classifiers were used: Logistic Regression (LR) [28], K-Nearest Neighbors (KNN) [11], Decision Tree (DT) [31] and Gradient Boosting (GBC) [19].

The implementation of the classifiers and standard oversampling algorithms was based on the Python libraries Scikit-Learn [30] and Imbalanced-Learn [26].

#### 5.4. Experimental procedure

In order to evaluate the performance of each combination of oversampler and classifier,  $n$ -fold cross validation was applied with  $n = 5$ . Let  $D$  one of the datasets. Before starting the training of the classifier, in each stage  $i \in \{1, 2, \dots, n\}$  of the  $n$ -fold cross validation procedure, synthetic data  $T_{g,i}$  were generated from the oversampler based on the training data  $T_i$  of the  $n - 1$  folds such that the resulting  $T_{g,i} \cup T_i$  training set becomes perfectly balanced. This enhanced training set in turn was used to train the classifier. The performance evaluation of the classifiers was done on the validation data  $V_i$  of the remaining fold, where  $V_i \cup T_i = D$  and  $V_i \cap T_i = \emptyset$ .

A variety of hyper-parameters were used for the oversamplers and classifiers. For SMOTE the optimal value of  $k$  nearest neighbors was selected as  $k \in \{3, 5\}$  while for G-SMOTE a hyper-parameter grid was generated from the Cartesian product of the three different selection strategies, the number of nearest neighbors  $k \in \{3, 5\}$ , the truncation factor  $\alpha_{trunc} \in \{-1.0, -0.5, 0.0, 0.25, 0.5, 0.75, 1.0\}$  and the deformation factor  $\alpha_{def} \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$ . As far the classifiers are concerned, the number of nearest neighbors for KNN was selected as  $k \in \{3, 5\}$ , the max depth for DT was selected as  $max\_depth \in \{3, 6\}$  while GBC hyper-parameter grid included the four combinations resulting from the Cartesian product of  $max\_depth \in \{3, 6\}$  and  $number\_of\_estimators \in \{50, 100\}$ .

Using the above hyper-parameters, the highest cross validation score for each combination of datasets, classifiers, oversamplers and evaluation metrics was reported. The experimental procedure was repeated 3 times and the reported results include the average values between the experiments. A ranking score was assigned to each oversampling method with the best and worst performing methods receiving scores equal to 1 and 4, respectively. We apply the Friedman test to confirm the statistical significance of the ranking results of the experiments [22]. The Friedman test is used to detect differences for a set of experimental attempts when normality assumption may not hold. In this case the null hypothesis represents the situation in which the classifiers show an identical performance, in terms of their mean ranking, independently of the oversampling method and performance metric used. Additionally, we apply the Holms test, using G-SMOTE as the control method [22]. The Holms test is a non-parametric version of the  $t$ -test, where the null hypothesis is whether the proposed G-SMOTE algorithm outperforms the other methods as the control method.

#### 5.5. Software implementation

As it was mentioned above, the implementation of the experimental procedure based on the Python programming language and Scikit-Learn/Imbalanced-Learn libraries. Specifically, the function that prepares and runs any comparative experiment can be found at [https://www.github.com/georgedouzas/scikit-learn-extensions/blob/master/sklearnnext/tools/imbalanced\\_analysis.py](https://www.github.com/georgedouzas/scikit-learn-extensions/blob/master/sklearnnext/tools/imbalanced_analysis.py), while the G-SMOTE implementation is available at [https://www.github.com/georgedouzas/scikit-learn-extensions/blob/master/sklearnnext/over\\_sampling/geometric\\_smote.py](https://www.github.com/georgedouzas/scikit-learn-extensions/blob/master/sklearnnext/over_sampling/geometric_smote.py). Being fully integrated with the Scikit-Learn ecosystem they can be adjusted and used for a variety of custom comparative studies. The experiments reported in this paper as well as the analysis of their results are reproducible using the scripts provided at <https://www.github.com/IMS-ML-Lab/publications>.

## 6. Results and discussion

In this section the performance of the different oversamplers is presented. The results are analyzed, by applying various statistical tests, to establish their significance. Additionally, a taxonomy of G-SMOTE hyper-parameters is presented as well as guidelines for their tuning relative to the characteristics of the imbalanced datasets.

### 6.1. Comparative presentation

The mean and standard error of cross validation scores across datasets for each combination of classifiers, metrics and oversamplers are presented in Table 2:

**Table 2**  
Results for mean cross validation scores of oversamplers across the datasets.

| Classifier | Metric  | NO OVERSAMPLING | RANDOM OVERSAMPLING | SMOTE       | G-SMOTE            |
|------------|---------|-----------------|---------------------|-------------|--------------------|
| LR         | AUC     | 0.90 ± 0.01     | 0.90 ± 0.01         | 0.90 ± 0.01 | <b>0.91 ± 0.01</b> |
| LR         | F-SCORE | 0.44 ± 0.05     | 0.59 ± 0.03         | 0.59 ± 0.03 | <b>0.60 ± 0.03</b> |
| LR         | G-MEAN  | 0.50 ± 0.05     | 0.84 ± 0.02         | 0.84 ± 0.02 | <b>0.86 ± 0.01</b> |
| KNN        | AUC     | 0.82 ± 0.02     | 0.82 ± 0.02         | 0.84 ± 0.02 | <b>0.86 ± 0.01</b> |
| KNN        | F-SCORE | 0.55 ± 0.03     | 0.58 ± 0.03         | 0.59 ± 0.03 | <b>0.62 ± 0.03</b> |
| KNN        | G-MEAN  | 0.63 ± 0.03     | 0.76 ± 0.03         | 0.78 ± 0.03 | <b>0.82 ± 0.02</b> |
| DT         | AUC     | 0.84 ± 0.01     | 0.85 ± 0.01         | 0.86 ± 0.01 | <b>0.89 ± 0.01</b> |
| DT         | F-SCORE | 0.62 ± 0.03     | 0.63 ± 0.03         | 0.64 ± 0.03 | <b>0.69 ± 0.02</b> |
| DT         | G-MEAN  | 0.73 ± 0.02     | 0.82 ± 0.01         | 0.83 ± 0.01 | <b>0.87 ± 0.01</b> |
| GBC        | AUC     | 0.91 ± 0.01     | 0.91 ± 0.01         | 0.91 ± 0.01 | <b>0.93 ± 0.01</b> |
| GBC        | F-SCORE | 0.66 ± 0.03     | 0.68 ± 0.03         | 0.70 ± 0.03 | <b>0.74 ± 0.02</b> |
| GBC        | G-MEAN  | 0.75 ± 0.02     | 0.82 ± 0.02         | 0.83 ± 0.01 | <b>0.87 ± 0.01</b> |

Table 2 shows that G-SMOTE systematically performs better on average than the rest of the methods. Particularly, the mean and standard error of percentage difference between G-SMOTE and SMOTE is presented in Table 3:

**Table 3**  
Results for percentage difference between G-SMOTE and SMOTE across the datasets.

| Classifier | Metric  | Difference      |
|------------|---------|-----------------|
| LR         | AUC     | 0.41 ± 0.07     |
| LR         | F-SCORE | 1.80 ± 0.43     |
| LR         | G-MEAN  | 2.15 ± 0.42     |
| KNN        | AUC     | 3.24 ± 0.92     |
| KNN        | F-SCORE | 13.00 ± 2.92    |
| KNN        | G-MEAN  | 149.69 ± 110.16 |
| DT         | AUC     | 3.51 ± 0.49     |
| DT         | F-SCORE | 12.60 ± 2.62    |
| DT         | G-MEAN  | 4.25 ± 0.63     |
| GBC        | AUC     | 1.65 ± 0.24     |
| GBC        | F-SCORE | 7.68 ± 1.35     |
| GBC        | G-MEAN  | 4.44 ± 0.54     |

As explained in Section 5.4 a ranking score in the range 1 to 4 is assigned to each oversampler. Then the mean and standard error of oversampling methods' ranking across the datasets for each combination of a classifier and evaluation metric is presented in Table 4:

**Table 4**  
Results for mean ranking of oversamplers across the datasets.

| Classifier | Metric  | NO OVERSAMPLING | RANDOM OVERSAMPLING | SMOTE       | G-SMOTE            |
|------------|---------|-----------------|---------------------|-------------|--------------------|
| LR         | AUC     | 2.91 ± 0.14     | 2.94 ± 0.10         | 2.74 ± 0.08 | <b>1.41 ± 0.07</b> |
| LR         | F-SCORE | 3.23 ± 0.14     | 2.88 ± 0.09         | 2.42 ± 0.10 | <b>1.47 ± 0.08</b> |
| LR         | G-MEAN  | 3.74 ± 0.08     | 2.47 ± 0.07         | 2.58 ± 0.08 | <b>1.21 ± 0.06</b> |
| KNN        | AUC     | 2.98 ± 0.10     | 3.39 ± 0.08         | 2.38 ± 0.09 | <b>1.25 ± 0.06</b> |
| KNN        | F-SCORE | 3.16 ± 0.15     | 2.73 ± 0.09         | 2.70 ± 0.10 | <b>1.41 ± 0.07</b> |
| KNN        | G-MEAN  | 3.70 ± 0.09     | 2.84 ± 0.07         | 2.17 ± 0.08 | <b>1.29 ± 0.06</b> |
| DT         | AUC     | 3.24 ± 0.10     | 3.16 ± 0.08         | 2.43 ± 0.10 | <b>1.17 ± 0.05</b> |
| DT         | F-SCORE | 2.80 ± 0.14     | 3.16 ± 0.08         | 2.66 ± 0.11 | <b>1.38 ± 0.07</b> |
| DT         | G-MEAN  | 3.65 ± 0.08     | 2.81 ± 0.08         | 2.37 ± 0.08 | <b>1.17 ± 0.05</b> |
| GBC        | AUC     | 2.99 ± 0.11     | 3.23 ± 0.09         | 2.62 ± 0.09 | <b>1.15 ± 0.05</b> |
| GBC        | F-SCORE | 3.20 ± 0.11     | 3.08 ± 0.08         | 2.53 ± 0.10 | <b>1.20 ± 0.05</b> |
| GBC        | G-MEAN  | 3.64 ± 0.08     | 2.92 ± 0.08         | 2.31 ± 0.06 | <b>1.12 ± 0.05</b> |

## 6.2. Statistical analysis

The results of the application of the Friedman test are shown in Table 5:

**Table 5**  
Results for Friedman test.

| Classifier | Metric  | p-value              | Significance |
|------------|---------|----------------------|--------------|
| LR         | AUC     | $2.3 \cdot 10^{-16}$ | True         |
| LR         | F-SCORE | $1.4 \cdot 10^{-16}$ | True         |
| LR         | G-MEAN  | $1.7 \cdot 10^{-30}$ | True         |
| KNN        | AUC     | $9.3 \cdot 10^{-25}$ | True         |
| KNN        | F-SCORE | $4.1 \cdot 10^{-16}$ | True         |
| KNN        | G-MEAN  | $2.1 \cdot 10^{-29}$ | True         |
| DT         | AUC     | $1.2 \cdot 10^{-25}$ | True         |
| DT         | F-SCORE | $8.3 \cdot 10^{-17}$ | True         |
| DT         | G-MEAN  | $3.8 \cdot 10^{-30}$ | True         |
| GBC        | AUC     | $1.6 \cdot 10^{-24}$ | True         |
| GBC        | F-SCORE | $5.6 \cdot 10^{-24}$ | True         |
| GBC        | G-MEAN  | $1.7 \cdot 10^{-32}$ | True         |

Therefore at a significance level of  $\alpha = 0.05$  the null hypothesis is rejected, i.e. the classifiers do not perform similarly in the mean rankings across the oversampling methods and evaluation metrics. Following the Friedman test, the Holm's method is applied to adjust the p-values of the paired difference test with G-SMOTE algorithm as the control method. The adjusted p-values are presented in Table 6:

**Table 6**  
Adjusted p-values using the Holm's method.

| Classifier | Metric  | NO OVERSAMPLING      | RANDOM OVERSAMPLING  | SMOTE                |
|------------|---------|----------------------|----------------------|----------------------|
| LR         | AUC     | $9.8 \cdot 10^{-4}$  | $2.8 \cdot 10^{-8}$  | $1.5 \cdot 10^{-7}$  |
| LR         | F-SCORE | $1.2 \cdot 10^{-9}$  | $2.8 \cdot 10^{-9}$  | $9.2 \cdot 10^{-5}$  |
| LR         | G-MEAN  | $1.5 \cdot 10^{-12}$ | $4.0 \cdot 10^{-8}$  | $1.4 \cdot 10^{-8}$  |
| KNN        | AUC     | $2.5 \cdot 10^{-9}$  | $2.1 \cdot 10^{-10}$ | $5.3 \cdot 10^{-5}$  |
| KNN        | F-SCORE | $1.1 \cdot 10^{-7}$  | $1.0 \cdot 10^{-9}$  | $2.0 \cdot 10^{-10}$ |
| KNN        | G-MEAN  | $2.7 \cdot 10^{-11}$ | $8.0 \cdot 10^{-5}$  | $1.6 \cdot 10^{-2}$  |
| DT         | AUC     | $1.8 \cdot 10^{-12}$ | $1.1 \cdot 10^{-13}$ | $1.3 \cdot 10^{-10}$ |
| DT         | F-SCORE | $1.6 \cdot 10^{-8}$  | $4.2 \cdot 10^{-15}$ | $1.8 \cdot 10^{-9}$  |
| DT         | G-MEAN  | $4.9 \cdot 10^{-12}$ | $4.9 \cdot 10^{-12}$ | $4.6 \cdot 10^{-10}$ |
| GBC        | AUC     | $5.7 \cdot 10^{-9}$  | $5.7 \cdot 10^{-9}$  | $1.6 \cdot 10^{-9}$  |
| GBC        | F-SCORE | $3.9 \cdot 10^{-12}$ | $4.1 \cdot 10^{-15}$ | $1.7 \cdot 10^{-12}$ |
| GBC        | G-MEAN  | $8.3 \cdot 10^{-12}$ | $1.0 \cdot 10^{-12}$ | $1.7 \cdot 10^{-12}$ |

Therefore, the null hypothesis of the Holm's test is rejected for all oversamplers at a significance level of  $\alpha = 0.05$ , indicating that the proposed method outperforms all other methods.

## 6.3. G-SMOTE taxonomy

The G-SMOTE hyper-parameters  $\alpha_{trunc}$ ,  $\alpha_{def}$  and  $\alpha_{sel}$ , called geometric hyper-parameters, allow to create a taxonomy of different hyper-parameters' configurations with a simple geometric interpretation. The various categories are tested during the experimental procedure and compared to the rest of the oversampling methods. A short description is provided below:

### – SMOTE algorithm.

The choice of hyper-parameters that reproduces the SMOTE algorithm is  $\alpha_{def} = 1.0$ ,  $\alpha_{trunc} = 1.0$  and  $\alpha_{sel} = \text{minority}$ . The first of them ensures that the initial hypersphere is deformed in to a line segment, the second truncates the half of the resulting line segment and the third chooses the SMOTE selection strategy.

### – Modified SMOTE algorithm.

The permissible data generation area remains a line segment as the above case. Therefore  $\alpha_{def} = 1.0$  but with no further restriction on the  $\alpha_{trunc}$  values. Also instead of the *minority* selection strategy, it uses either the *majority* or *combined* one. This case can be considered as a modification of the SMOTE mechanism where the line segment is either truncated, expanded or rotated.



- Pure G-SMOTE algorithm.

This includes any hyper-parameters' configuration where the permissible data generation area is expanded from a line segment of the above cases into a hyper-spheroid. The choice of hyper-parameters in this case is  $\alpha_{def} < 1.0$  with no further restriction on the rest of the geometrical hyper-parameters.

Table 7 shows the frequency of the above 3 cases as an optimal configuration across the total 26,391 experiments:

**Table 7**  
Optimal configuration frequency for the three G-SMOTE cases.

| Case              | Frequency |
|-------------------|-----------|
| Pure G-SMOTE      | 23,743    |
| Modified SMOTE    | 2,392     |
| SMOTE             | 256       |
| Total experiments | 26,391    |

Therefore pure G-SMOTE represents the configuration with the highest frequency and ranking among the 3 cases, covering the majority of optimal hyper-parameter configurations. Additionally, SMOTE represents a small fraction of the optimal cases while its modification improves significantly the original data generation mechanism.

#### 6.4. Analysis and tuning of optimal hyper-parameters

As it was explained on the above section, the G-SMOTE geometric hyper-parameters control the characteristics of the data generation process. More specifically, the *majority* selection strategy expands aggressively the minority class area, increasing the variety of generated samples. Lower absolute values of the truncation and deformation factors have a similar effect. On the other hand, the *minority* selection strategy and higher absolute values of the truncation and deformation factors result in a data generation process that resembles the SMOTE mechanism.

An analysis of the G-SMOTE geometric hyper-parameters dependence on Imbalance Ratio (IR) and number of samples over number of features ratio (R) shows the following:

- High IR or low R.

The *majority* (or *combined*) selection strategy and lower absolute values of the truncation and deformation factors dominate as optimal hyper-parameters.

- Low IR or high R.

The *minority* selection strategy and higher absolute values of the truncation and deformation factors dominate as optimal hyper-parameters.

Therefore datasets with higher IR or lower R values favor the generation of diverse synthetic samples. The explanation for this finding is that for high IR values the SMOTE data generation process creates either nearly duplicate or noisy examples, while in sparse input spaces, where R is low, the linear interpolation mechanism of SMOTE generates samples on a special direction of the input space that effectively makes them nearly duplicate or noisy. Contrary to the above, for lower IR or higher R values, the input space contains enough information to avoid the above shortcomings of the SMOTE mechanism.

## 7. Conclusions

In this paper we presented G-SMOTE, a new oversampling algorithm, that extends the SMOTE data generation mechanism. G-SMOTE selects a safe radius around each minority class instance and generates artificial data within a (truncated) hyper-spheroid. G-SMOTE performance was evaluated on 69 datasets with different imbalance ratios and compared to no oversampling, Random Oversampling and SMOTE, using Logistic Regression, K-Nearest Neighbors, Decision Tree and Gradient Boosting Machine as classifiers.

The results show that G-SMOTE performs significantly better compared to the other methods. The explanation for this improvement in performance relates to the ability of G-SMOTE to generate artificial data in safe areas of the input space, while, at the same time, aggressively increasing the diversity of the generated instances. G-SMOTE parametrizes efficiently the data generation process and adapts to the special characteristics of each imbalanced dataset. We make available an implementation of G-SMOTE at <https://www.github.com/georgedouzas/scikit-learn-extensions/blob/master/sklearnnext/oversampling/geometricismote.py>.

G-SMOTE can be a useful tool for researchers and practitioners since it results in the generation of high quality artificial data and only requires the tuning of a small number of parameters.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

- [1] J. Alcalá-Fdez, A. Fernández, J. Luengo, J. Derrac, S. García, L. Sánchez, F. Herrera, Keel data-mining software tool: data set repository, integration of algorithms and experimental analysis framework, *J. Mult.-Valued Log. Soft Comput.* 17 (2011).
- [2] S. Barua, M.M. Islam, X. Yao, K. Murase, MWMOTE - majority weighted minority oversampling technique for imbalanced data set learning, *IEEE Trans. Knowl. Data Eng.* 26 (2) (2014) 405–425, doi:[10.1109/TKDE.2012.232](https://doi.org/10.1109/TKDE.2012.232).
- [3] G.E.A.P.A. Batista, R.C. Prati, M.C. Monard, A study of the behavior of several methods for balancing machine learning training data, *J. Mach. Learn. Res.* 6 (1) (2004) 20–29, doi:[10.1145/1007730.1007735](https://doi.org/10.1145/1007730.1007735).
- [4] R. Blagus, L. Lusa, SMOTE For high-dimensional class-imbalanced data, *BMC Bioinform.* 14 (1) (2013) 106, doi:[10.1186/1471-2105-14-106](https://doi.org/10.1186/1471-2105-14-106).
- [5] C. Bunkhumpornpat, K. Sinapiromsaran, C. Lursinsap, Safe-level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem, in: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 5476 LNAI, 2009, pp. 475–482, doi:[10.1007/978-3-642-01307-2\\_43](https://doi.org/10.1007/978-3-642-01307-2_43).
- [6] C. Bunkhumpornpat, K. Sinapiromsaran, C. Lursinsap, DBSMOTE: Density-based synthetic minority over-sampling technique, *Appl. Intell.* 36 (3) (2012) 664–684, doi:[10.1007/s10489-011-0287-y](https://doi.org/10.1007/s10489-011-0287-y).
- [7] N.V. Chawla, Data mining for imbalanced datasets: an overview, in: *Data Mining and Knowledge Discovery Handbook*, Springer-Verlag, 2005, pp. 853–867, doi:[10.1007/0-387-25465-x\\_40](https://doi.org/10.1007/0-387-25465-x_40).
- [8] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, *J. Artif. Intell. Res.* 16 (2002) 321–357, doi:[10.1613/jair.953](https://doi.org/10.1613/jair.953).
- [9] N.V. Chawla, A. Lazarevic, L. Hall, K. Boyer, SMOTEBoost: improving prediction of the minority class in boosting, *Princ. Knowl. Discov. Databases, PKDD-2003* (2003) 107–119, doi:[10.1007/b13634](https://doi.org/10.1007/b13634).
- [10] D.A. Cieslak, N.V. Chawla, A. Striegel, Combating imbalance in network intrusion datasets, in: *2006 IEEE International Conference on Granular Computing*, 2006, pp. 732–737, doi:[10.1109/GRC.2006.1635905](https://doi.org/10.1109/GRC.2006.1635905).
- [11] T. Cover, P. Hart, Nearest neighbor pattern classification, *IEEE Trans. Inf. Theory* 13 (1) (1967) 21–27, doi:[10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).
- [12] A. DasGupta, *Probability for statistics and machine learning*, Springer New York, 2011, doi:[10.1007/978-1-4419-9634-3](https://doi.org/10.1007/978-1-4419-9634-3).
- [13] P. Domingos, Metacost: a general method for making classifiers cost-sensitive, *Proceed Fifth ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.* 55 (1999) 155–164, doi:[10.1145/312129.312220](https://doi.org/10.1145/312129.312220).
- [14] G. Douzas, F. Bacao, Effective data generation for imbalanced learning using conditional generative adversarial networks, *Expert Syst. Appl.* (2017), doi:[10.1016/j.eswa.2017.09.030](https://doi.org/10.1016/j.eswa.2017.09.030).
- [15] G. Douzas, F. Bacao, Self-organizing map oversampling (somo) for imbalanced data set learning, *Expert Syst. Appl.* 82 (2017) 40–52, doi:[10.1016/j.eswa.2017.03.073](https://doi.org/10.1016/j.eswa.2017.03.073).
- [16] G. Douzas, F. Bacao, F. Last, Improving imbalanced learning through a heuristic oversampling method based on k-means and smote, *Inf. Sci. (N.Y.)* 465 (2018) 1–20, doi:[10.1016/j.ins.2018.06.056](https://doi.org/10.1016/j.ins.2018.06.056).
- [17] A. Fernández, S. García, F. Herrera, N.V. Chawla, Smote for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary, *J. Artif. Intell. Res.* 61 (1) (2018) 863–905.
- [18] A. Fernández, V. López, M. Galar, M.J. del Jesus, F. Herrera, Analysing the classification of imbalanced data-sets with multiple classes: binarization techniques and ad-hoc approaches, *Knowl. Based Syst.* 42 (2013) 97–110, <https://doi.org/10.1016/j.knsys.2013.01.018>.
- [19] J.H. Friedman, Greedy function approximation: a gradient boosting machine, *Ann. Stat.* 29 (5) (2001) 1189–1232, doi:[10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451).
- [20] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, F. Herrera, A review on ensembles for the Class imbalance problem: bagging-, boosting-, and hybrid-based approaches, 2012, doi:[10.1109/TSMCC.2011.2161285](https://doi.org/10.1109/TSMCC.2011.2161285).
- [21] H. Guo, H.L. Viktor, Learning from imbalanced data sets with boosting and data generation : the databoost-IM approach, *ACM SIGKD Explor. Newsl. Spec. Issue Learn. Imbalance. Dataset.* 6 (1) (2004) 30–39, doi:[10.1145/1007730.1007736](https://doi.org/10.1145/1007730.1007736).
- [22] I. Guyon, *Design of experiments of the NIPS 2003 variable selection benchmark, NIPS 2003 workshop on feature extraction*, 2003.
- [23] H. Han, W.-Y. Wang, B.-H. Mao, Borderline-SMOTE: anew over-sampling method in imbalanced data sets learning, in: *Lecture Notes in Computer Science*, Springer Berlin Heidelberg, 2005, pp. 878–887, doi:[10.1007/11538059\\_91](https://doi.org/10.1007/11538059_91).
- [24] H. He, E.A. García, Learning from imbalanced data, *IEEE Trans. Knowl. Data Eng.* 21 (9) (2009) 1263–1284, doi:[10.1109/TKDE.2008.239](https://doi.org/10.1109/TKDE.2008.239).
- [25] T. Jo, N. Japkowicz, Class imbalances versus small disjuncts, *ACM SIGKD Explor. Newsl.* 6 (1) (2004) 40, doi:[10.1145/1007730.1007737](https://doi.org/10.1145/1007730.1007737).
- [26] G. Lemaitre, F. Nogueira, C.K. Aridas, Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning, *J. Mach. Learn. Res.* 18 (2016) 1–5, <http://www.jmlr.org/papers/volume18/16-365/16-365.pdf>.
- [27] M. Lichman, *Uci machine learning repository*, 2013.
- [28] P. McCullagh, J. Nelder, *Generalized Linear Models*, Chapman and Hall/CRC Monographs on Statistics and Applied Probability Series Chapman & Hall, Second Edition, 1989.
- [29] I. Nekooimehr, S.K. Lai-Yuen, Adaptive semi-supervised weighted oversampling (A-SUWO) for imbalanced datasets, *Expert Syst. Appl.* 46 (2016) 405–416, doi:[10.1016/j.eswa.2015.10.031](https://doi.org/10.1016/j.eswa.2015.10.031).
- [30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-Learn: machine Learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830, doi:[10.1007/s13398-014-0173-7](https://doi.org/10.1007/s13398-014-0173-7).
- [31] S.L. Salzberg, C4.5: Programs for machine learning by j. ross quinlan. morgan kaufmann publishers, inc., 1993, *Mach. Learn.* 16 (3) (1994) 235–240, doi:[10.1007/BF00993309](https://doi.org/10.1007/BF00993309).
- [32] Z. Sun, Q. Song, X. Zhu, H. Sun, B. Xu, Y. Zhou, A novel ensemble method for classifying imbalanced data, *Pattern Recognit.* 48 (5) (2015) 1623–1637, doi:[10.1016/j.patcog.2014.11.014](https://doi.org/10.1016/j.patcog.2014.11.014).
- [33] B. Tang, H. He, KernelADASYN: Kernel based adaptive synthetic data generation for imbalanced learning, 2015 IEEE Congress on Evolutionary Computation, IEEE, 2015, doi:[10.1109/cec.2015.7256954](https://doi.org/10.1109/cec.2015.7256954).
- [34] K.M. Ting, An instance-weighting method to induce cost-sensitive trees, *IEEE Trans. Knowl. Data Eng.* 14 (3) (2002) 659–665, doi:[10.1109/TKDE.2002.1000348](https://doi.org/10.1109/TKDE.2002.1000348).
- [35] P.M. Vaidya, An o(n log n) algorithm for the all-nearest-neighbors problem, *Discret. Comput. Geom.* 4 (2) (1989) 101–115, doi:[10.1007/bf02187718](https://doi.org/10.1007/bf02187718).
- [36] S. Wang, L.L. Minku, X. Yao, Resampling-based ensemble methods for online class imbalance learning, *IEEE Trans. Knowl. Data Eng.* 27 (5) (2015) 1356–1368, doi:[10.1109/TKDE.2014.2345380](https://doi.org/10.1109/TKDE.2014.2345380).
- [37] D.L. Wilson, Asymptotic properties of nearest neighbor rules using edited data, *IEEE Trans. Syst. Man Cybern.* 2 (3) (1972) 408–421, doi:[10.1109/TSMC.1972.4309137](https://doi.org/10.1109/TSMC.1972.4309137).